

HACKATHON PROJECT

InboxIQ

Intelligent message triage & notification routing

NOTIFY

DIGEST

MUTE

Live app: inbox-iq-hack.vercel.app

API: inboxiq-9903.onrender.com

THE PROBLEM

Notification overload is drowning out what matters

80+

notifications/day

typical active messaging user

3

message classes

personal, business, group — all mixed

1

inbox

no built-in priority signal

Payments, urgent alerts, group chatter, promos, and scams all compete for the same attention — without smart filtering, users either miss what matters or drown it out entirely.

Route every message to the right attention tier



NOTIFY

Surface immediately

Payments · Security · Urgent personal



DIGEST

Bundle into a periodic summary

Business updates · Group activity



MUTE

Suppress entirely

Promotions · Spam · Low-value forwards

Three layers, one pipeline

1

Prediction Pipeline

`code/main.py` + `engines/` — standalone batch classifier over the CSV dataset

2

Backend API

FastAPI service (`code/backend/`) exposing live predictions & analytics over HTTP

3

Frontend Dashboard

React + TS + Vite SPA (`code/frontend/`) — Dashboard, Messages, Analytics, Inspector, Command Center

Built with a lean, modern stack

Frontend

- React + TypeScript
- Vite
- Tailwind CSS
- Recharts
- Framer Motion

Backend

- Python
- FastAPI
- Pydantic
- Uvicorn

Data & Hosting

- CSV-based dataset
- Vercel (frontend)
- Render (backend)

A modular reasoning pipeline



reasoning_engine.py

Primary decision logic



confidence_engine.py

Scores prediction confidence



scam_detector.py

Flags scam risk



spam_detector.py

Flags spam / low value



business_engine.py

Business-context rules



group_engine.py

Group-context rules



context_loader.py

Loads & normalizes dataset

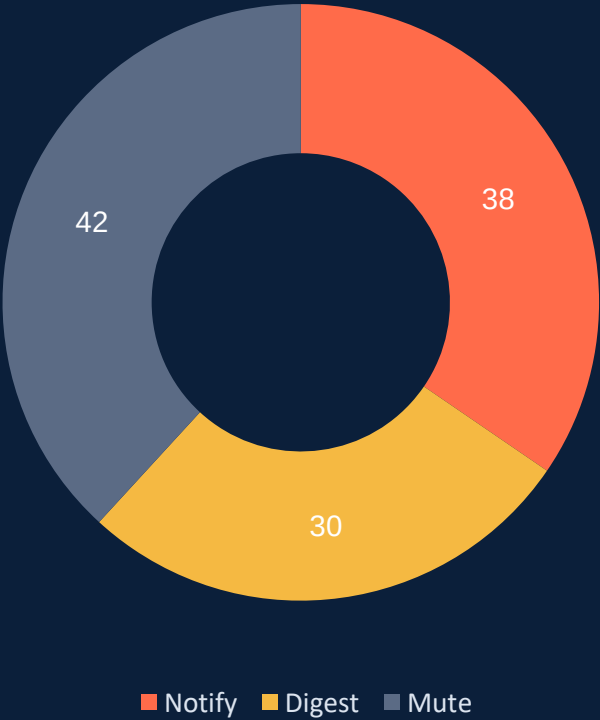


notification_router.py

Final routing decision

110 messages classified end-to-end

Action Distribution



Message type breakdown



A full analytics dashboard on top of the engine

1

Dashboard

Top-line stats & recent activity

2

Messages

Filterable, paginated message list

3

Analytics

Sender / group / business breakdowns

4

Inspector

Per-message reasoning trace

5

Command Center

System-health style gauges

6

Business / Groups

Directory & display-name lookups

Hardened for production before shipping

Check	Before	After
Unsafe `any` types	15 instances	0 — replaced with real interfaces
ESLint (--max-warnings 0)	Failing	0 errors, 0 warnings
TypeScript build	Failing (missing vite-env.d.ts)	Clean tsc --noEmit
Production build	N/A	npm run build succeeds
Backend endpoints	Unverified	All 12 endpoints curl-tested

Split-stack deploy: Vercel + Render

Frontend — Vercel

Root Directory `code/frontend`

Framework `Vite (auto-detected)`

Env Var `VITE_API_URL`

Live: inbox-iq-hack.vercel.app

Backend — Render

Root Directory `code/backend`

Build Command `pip install -r requirements.txt`

Start Command `uvicorn main:app --port $PORT`

Env Var `PYTHON_VERSION=3.11.9`

Live: inboxiq-9903.onrender.com

CHALLENGES

Real deploy friction, debugged live



pydantic-core failed to build on Python 3.14

Pinned an older Python with prebuilt wheels (3.12 locally, 3.11.9 on Render)



macOS blocked global pip installs

Used a proper virtual environment



Render kept losing requirements.txt

Diagnosed with pwd/ls/find, then aligned Root Directory & commands



Frontend deployed but showed no data

VITE_API_URL wasn't set — added it and redeployed

InboxIQ is live end-to-end

Routing engine → FastAPI backend → React dashboard, deployed and verified.

Frontend

inbox-iq-hack.vercel.app

API

inboxiq-9903.onrender.com

Repo

github.com/Yashhh710/InboxIQ

Thank you